

Ejercicio 1

Pregunta teórica

Podemos usar expresiones lambda cuando una interfaz tiene un único método abstracto.

Este tipo de interfaces se llaman interfaces funcionales.

El criterio para saber si una interfaz puede mapearse con una lambda es comprobar que solo tenga un método abstracto. Puede tener métodos default o static sin problema.

La interfaz List NO puede usarse con expresiones lambda porque tiene muchos métodos abstractos (add, remove, get, etc.), por lo tanto no es una interfaz funcional.

Ejercicio 2

Sí, la interfaz `java.util.concurrent.Callable` puede representarse mediante una expresión lambda porque únicamente tiene un método abstracto:

`V call() throws Exception;`

Como solo posee un método abstracto, es una interfaz funcional y Java permite usar lambdas.

Ejemplo:

```
Callable<String> tarea = () -> "Hola";
```

Ejercicio 3

Sí, `java.util.Comparator` también puede utilizar expresiones lambda porque únicamente tiene un método abstracto:

```
compare(T o1, T o2)
```

Ejemplo:

```
Comparator<Integer> comparador =
```

```
(a, b) -> a - b;
```

Ejercicio 4

Clase anónima transformada a lambda

```
Runnable runnable = () -> {  
    String texto = "Vamos a crear un array";  
    String[] partes = texto.split(" ");  
    for (String parte : partes) {  
        System.out.println(parte);  
    }  
};
```

Código de prueba

```
public class Main {  
    public static void main(String[] args) {  
        Runnable runnable = () -> {  
            String texto = "Vamos a crear un array";  
            String[] partes = texto.split(" ");  
            for (String parte : partes) {  
                System.out.println(parte);  
            }  
        };  
        runnable.run();  
    }  
}
```

Ejercicio 5

Método convertido a lambda

```
Function<String, String> caracteresPares = (texto) -> {  
    StringBuilder sb = new StringBuilder();  
    for (int i = 0; i < texto.length(); i++) {  
        if (i % 2 == 0) {  
            sb.append(texto.charAt(i));  
        }  
    }  
    return sb.toString();  
};
```

Ejercicio 6

Ejecutar la función con "1234567890"

```
public class Main {  
    public static void main(String[] args) {  
        Function<String, String> caracteresPares = (texto) -> {  
            StringBuilder sb = new StringBuilder();  
            for (int i = 0; i < texto.length(); i++) {  
                if (i % 2 == 0) {  
                    sb.append(texto.charAt(i));  
                }  
            }  
            return sb.toString();  
        };  
        String resultado = caracteresPares.apply("1234567890");  
        System.out.println(resultado);  
    }  
}
```

Ejercicio 7

Método paresEjecuta

```
import java.util.function.Function;

public class Main {

    public static String paresEjecuta(
        Function<String, String> funcion,
        String texto) {
        return funcion.apply(texto);
    }

    public static void main(String[] args) {
        Function<String, String> caracteresPares = (texto) -> {
            StringBuilder sb = new StringBuilder();
            for (int i = 0; i < texto.length(); i++) {
                if (i % 2 == 0) {
                    sb.append(texto.charAt(i));
                }
            }
            return sb.toString();
        };

        String resultado =
            paresEjecuta(caracteresPares, "1234567890");

        System.out.println(resultado);
    }
}
```

Ejercicio 8

Supplier sin return

```
Supplier<String> prg =  
    () -> "Estoy en clase de programación";
```

Supplier usando return

```
Supplier<String> prg = () -> {  
    return "Estoy en clase de programación";  
};
```

Pregunta teórica

No es obligatorio usar return cuando la lambda tiene una única expresión.

Si usamos llaves { }, entonces sí debemos utilizar return.

Ejercicio 9

Ejecutar Supplier

```
public class Main {  
    public static void main(String[] args) {  
        Supplier<String> prg =  
            () -> "Estoy en clase de programación";  
        String textoPrg = prg.get();  
        System.out.println(textoPrg);  
    }  
}
```

Ejercicio 10

Lista de ciudades ordenadas y con mayúscula

```
import java.util.Arrays;
import java.util.List;

public class Main {

    public static void main(String[] args) {

        List<String> ciudades = Arrays.asList(

            "elche",

            "alicante",

            "elda",

            "petrer",

            "aspe",

            "crevillente",

            "alcoy",

            "ibi",

            "benidorm",

            "matola"

        );

        ciudades.stream()

            .sorted((a, b) -> a.compareTo(b))

            .map(ciudad ->

                ciudad.substring(0, 1).toUpperCase()

                + ciudad.substring(1))

            .forEach(ciudad ->

                System.out.println(ciudad));

    }

}
```